CAPSTONE PROJECT · VIVA

CloudLens

DevOps dashboard for resource and cost monitoring across AWS, Azure and GCP

AWS | Azure | GCP | FastAPI · PostgreSQL · React · Prometheus · Docker

[Student name] · [Roll no.] · [Institute] · October 2026

PROBLEM STATEMENT

Three clouds, three consoles, no single picture

Fragmented data

Each provider names services, tags and regions differently, so costs can’t be compared directly.

Late surprises

Runaway jobs and budget overruns are noticed on the invoice, weeks after the money is spent.

Hidden waste

Idle and over-provisioned resources keep billing because nobody links cost to utilisation.

Goal: one centralised dashboard that aggregates all three clouds, tracks utilisation and cost trends, sets thresholds and alerts on anomalies.

PROJECT GOALS

Four goals, all delivered

01

Aggregate AWS, Azure and GCP

Three connectors plus a mock mode, one unified schema

02

Real-time and historical views

5-minute utilisation, trends, month-over-month, anomalies

03

Customisable alerting

8 rule types via Prometheus Alertmanager to Slack and Teams

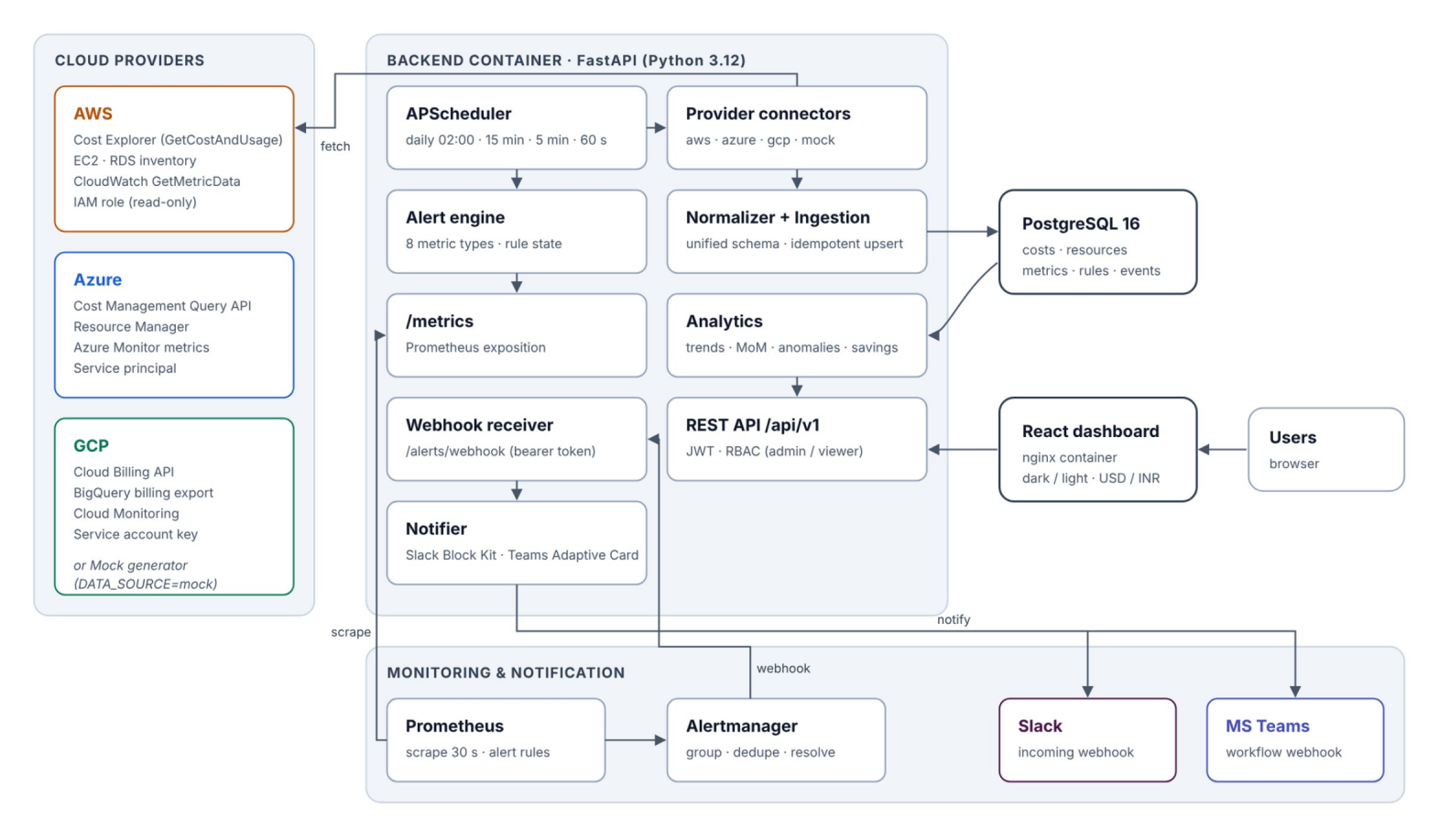
04

Filters and drill-down

Provider, environment, resource type, date range, per resource

ARCHITECTURE

Six containers, one Docker Compose file



TECHNOLOGY CHOICES

Each tool chosen for a reason

Layer	Choice	Why
Database	PostgreSQL 16	Relational cost data, idempotent upserts, indexed GROUP BY
Backend	Python FastAPI	Validation, OpenAPI docs, async-capable
Scheduling	APScheduler	In-process jobs, no extra broker to run or pay for
Frontend	React + Recharts	Filters, drill-down, RBAC, dark/light, USD/INR
Alerting	Prometheus + Alertmanager	Industry standard grouping, dedupe, resolve
Packaging	Docker Compose → EC2	Same stack on a laptop and in staging

DATA AGGREGATION

One schema for every cloud

Field	AWS Cost Explorer	Azure Cost Mgmt	GCP billing export
day	TimePeriod.Start	UsageDate	usage_start_time
service	SERVICE	ServiceName	service.description
environment	environment$prod	tag value	label value
cost (USD)	UnblendedCost	Cost, INR → USD	cost + credits

Normaliser maps services to 6 types, tags to 4 environments

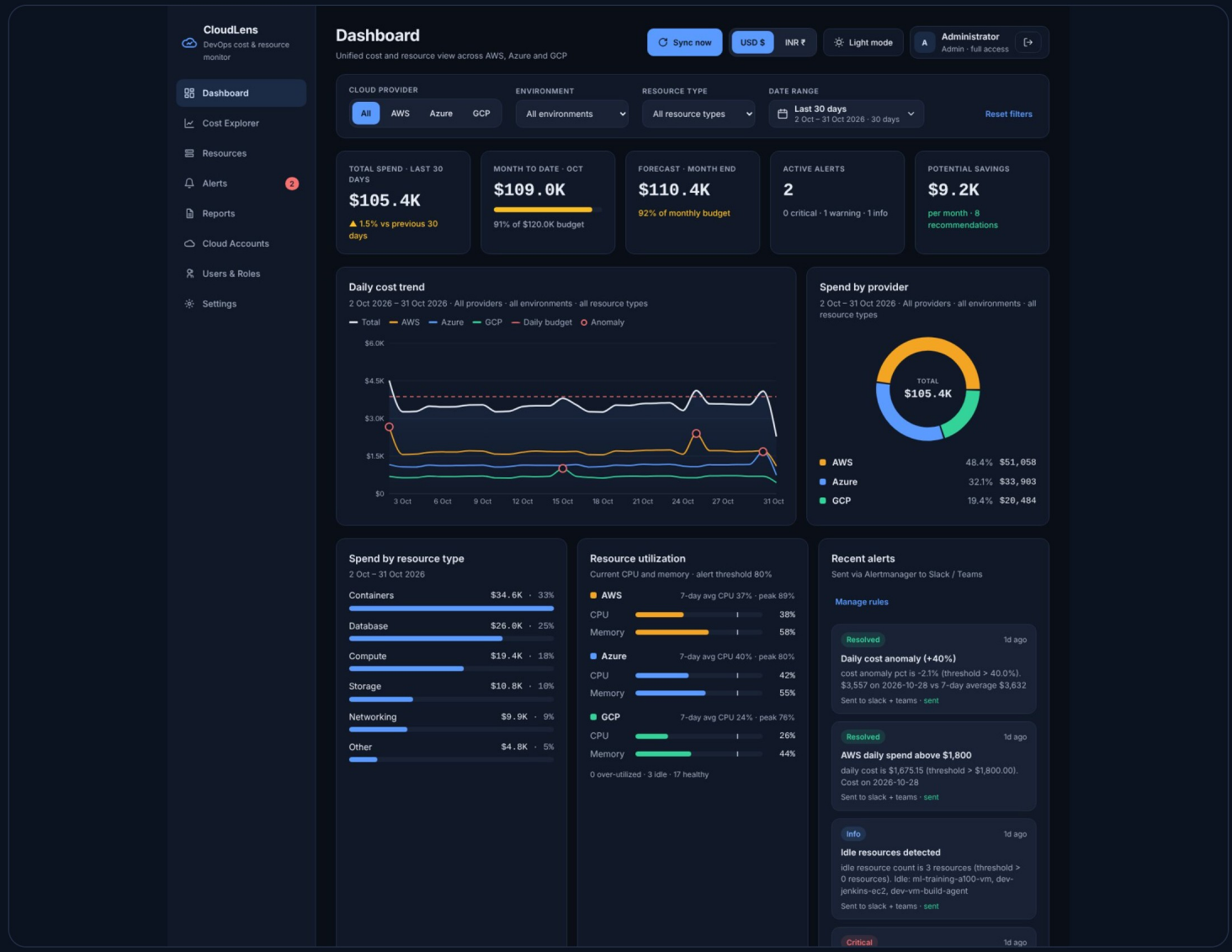
Idempotent upsert on the natural key: re-runs never duplicate

Scheduled: daily 02:00, cost 15 min, metrics 5 min

REAL-TIME DASHBOARD

Everything on one screen

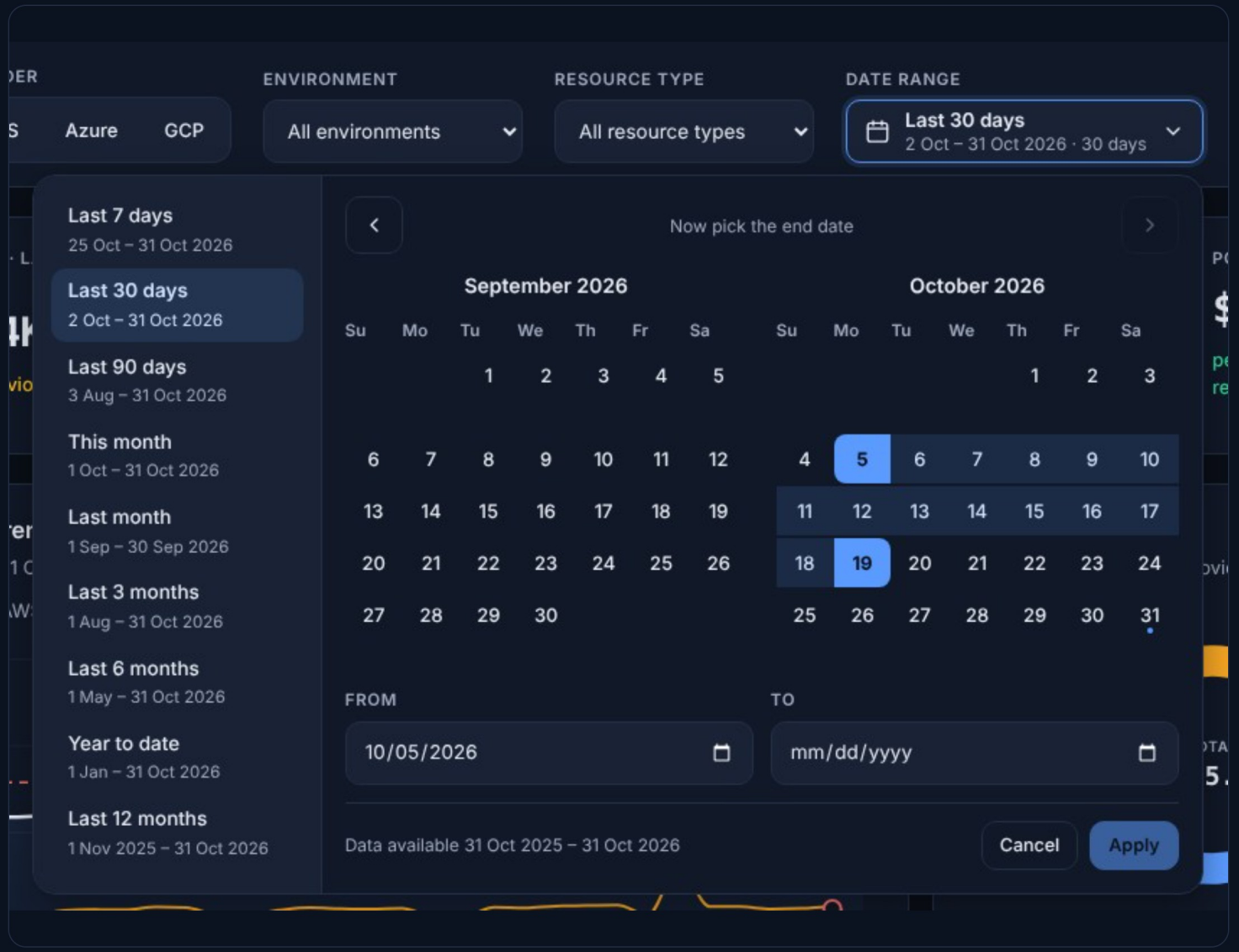
- KPIs: spend, MTD vs budget, forecast, alerts, savings
- Trend with budget line and anomaly markers
- Utilisation per cloud, 80% threshold
- Date-range calendar shared by every screen
- Auto-refresh every 60 s



HISTORICAL ANALYSIS

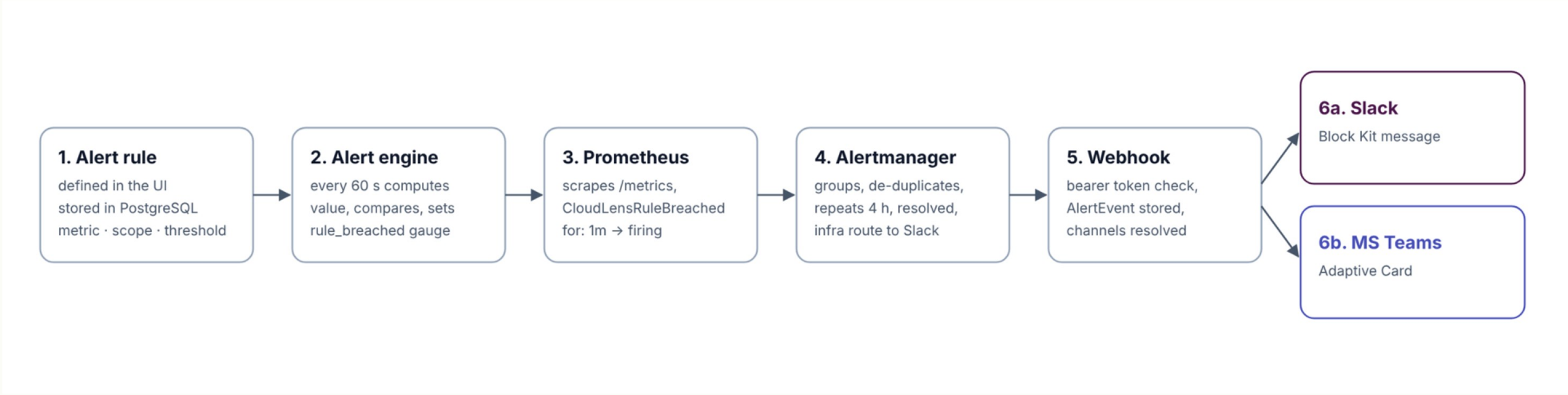
Any period, validated

- Presets from today: 7–90 days, months, YTD
- Custom range: click start, click end, Apply
- No future dates; start ≤ end; ≤ 731 days
- Month-over-month with run-rate for partial months
- Reports for any range, e.g. last 6 months, CSV



ALERTING PIPELINE

Rules in the UI, delivery by Prometheus



One gauge per rule: **cloudlens_alert_rule_breached**

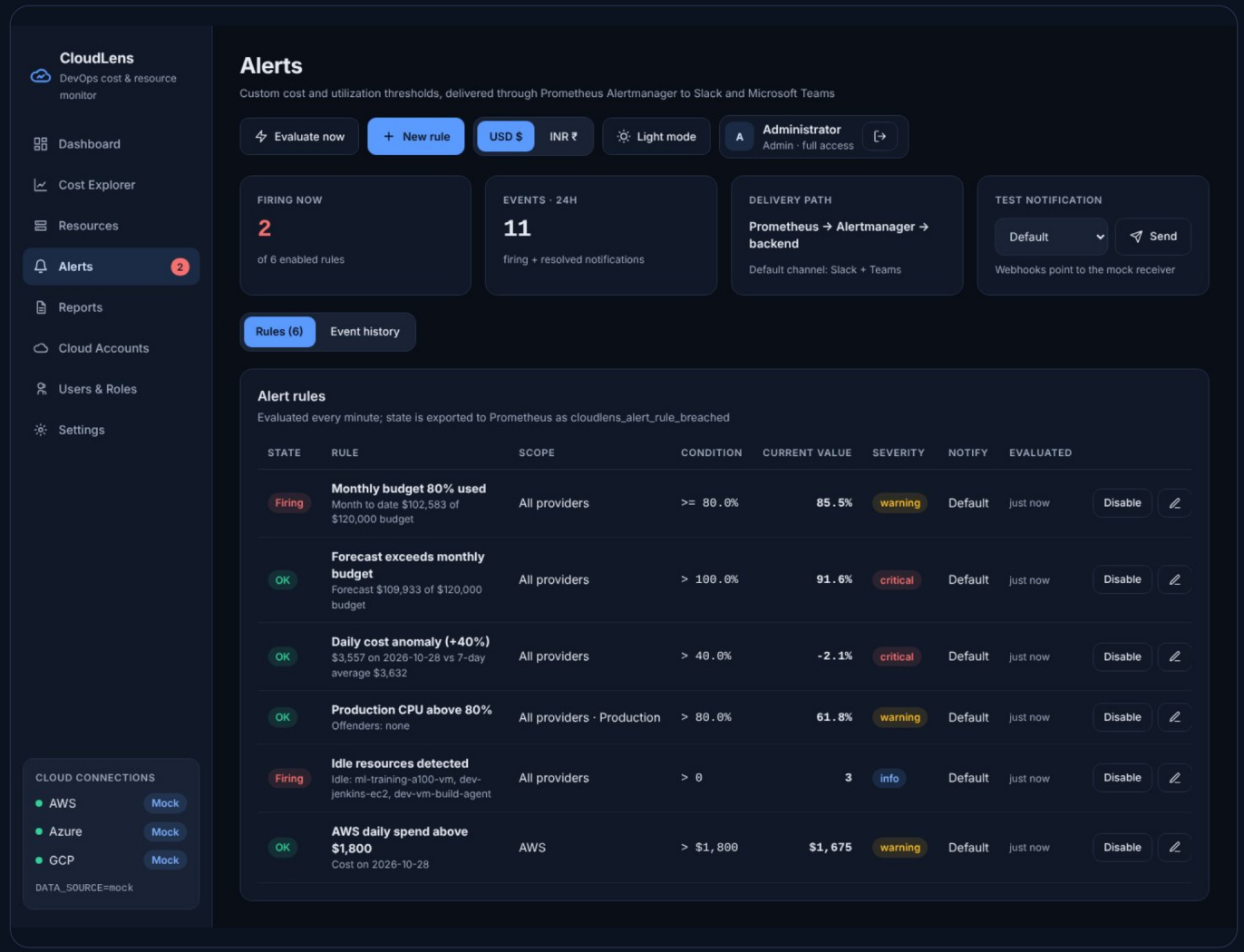
Grouping, dedupe, 4 h repeat, **resolved** messages

Webhook protected by a **bearer token**

CUSTOMISABLE ALERTS

8 rule types, any scope

Watch	Example
Daily cost	AWS > $1,800
Budget used %	≥ 80% warning
Forecast %	> 100% critical
Cost anomaly %	> 40% critical
CPU / memory	Prod CPU > 80%
Idle resources	count > 0



COST OPTIMISATION

Monitoring that pays for itself

$9.2K
monthly savings found on demo data (about 8.5% of spend)

$0.60
per month in Cost Explorer calls: 3-day lookback, calls counted per sync

~$21
per month to run the whole stack on one t3.small

Recommendations: stop idle, rightsize, off-hours schedules, storage tiering. Plus budget forecasts, anomaly alerts, retention policies and partition-filtered BigQuery queries.

TESTING

Verified at every layer

84
backend tests pass on SQLite and PostgreSQL

85%
line coverage, analytics 97%

31
frontend unit tests, incl. date picker

8
Slack + Teams messages in the end-to-end run

- Breakdowns always sum to the period total; re-syncs never change row counts
- Future, reversed and > 731-day ranges rejected (HTTP 422)
- Real Prometheus + Alertmanager: fire, deliver, resolve
- Every screen captured in both themes and at phone width

DEPLOYMENT

Laptop to EC2 staging in three commands

```
$ cp .env.example .env.staging
$ KEY_NAME=my-key ./deploy/aws/create-ec2.sh
$ EC2_HOST=<ip> ./deploy/aws/deploy.sh
Healthy. Open http://<ip>/
```

- t3.small, IMDSv2, 20 GB gp3, swap
- IAM instance role, read-only policy
- Only port 80 public; SSH from your IP
- Non-root containers, health checks, log rotation
- Backups, hardening checklist

LIVE DEMO

Five minutes, end to end

#	Show	Point to make
1	Dashboard, filter AWS → Production	One view, every widget follows the filter
2	Toggle INR and light mode	Lakh / crore units; theme persists
3	Click a resource	Drill-down: cost bars, CPU, recommendation
4	Date picker → Last 6 months; type a future date	Presets from today; validation blocks future / reversed ranges
5	Inject an AWS spike, wait ~90 s	Prometheus fires; Slack + Teams in mock inbox
6	Sign in as viewer	Admin actions hidden and blocked by the API

CONCLUSION

What we learned, and what’s next

Learnings

- Each cloud exposes cost differently; a unified contract hides it
- Idempotent upserts make scheduled jobs safe to re-run
- Paid APIs need cost-aware design and call accounting
- Keep users in the UI, keep alert delivery standard

Future scope

- Alembic migrations, HTTPS, live exchange rate
- Separate worker; TimescaleDB for metrics
- ML forecasting, per-service anomalies
- SSO, audit log, scheduled email reports

Thank you

Questions?

`docs/ · report · 15 module docs · 10 screen docs` `/docs → live API reference`

[Student name] · [Roll no.] · [Institute]